

EXPERIMENT 4

Name- Niraj Kothawade

Class-D20A

Roll No- 30

Aim:Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

Primitive Data Types

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- `uint` / `int`: unsigned and signed integers of different sizes (e.g., `uint256`, `int128`).
- `bool`: represents logical values (true or false).
- `address`: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- `bytes` / `string`: store binary data or textual data.

Variables in Solidity can be state variables (stored on the blockchain permanently), local variables (temporary, created during function execution), or global variables (special predefined variables such as `msg.sender`, `msg.value`, and `block.timestamp`).

Functions allow execution of contract logic. Special types of functions include:

- `pure`: cannot read or modify blockchain state; they work only with inputs and internal computations.
- `view`: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability

and debugging.

3. Visibility, Modifiers and Constructors

- Function Visibility defines who can access a function:
 - public: available both inside and outside the contract.
 - private: only accessible within the same contract.
 - internal: accessible within the contract and its child contracts.
 - external: can be called only by external accounts or other contracts.
- 4. Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- 5. Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- if-else allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- Loops (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- Arrays: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- Mappings: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.

- Structs: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- Enums: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

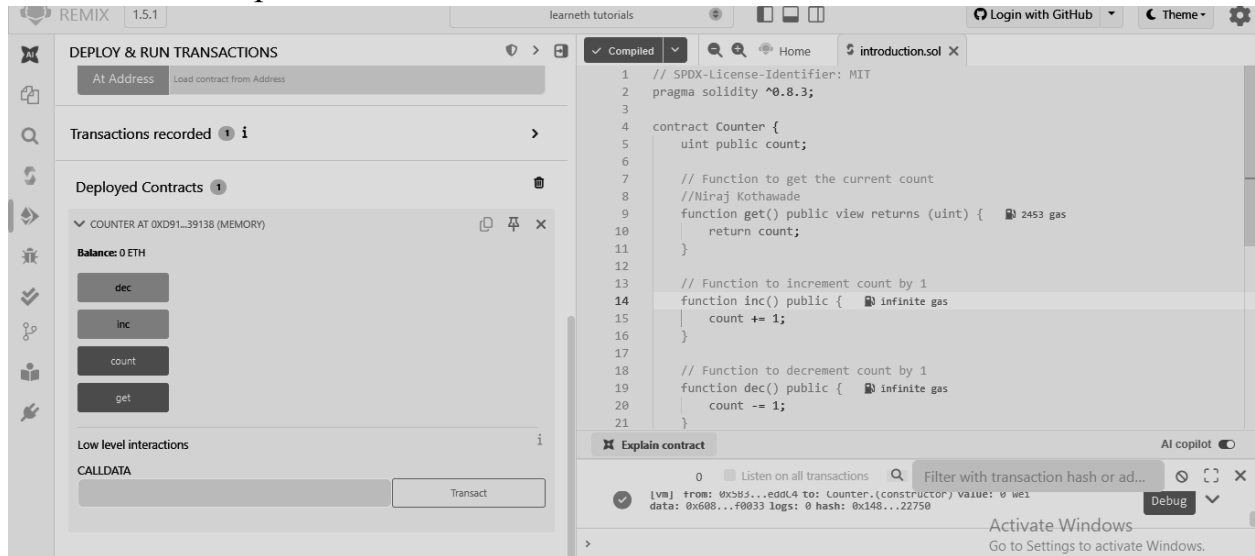
- storage: Data stored permanently on the blockchain. Examples: state variables.
- memory: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- calldata: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

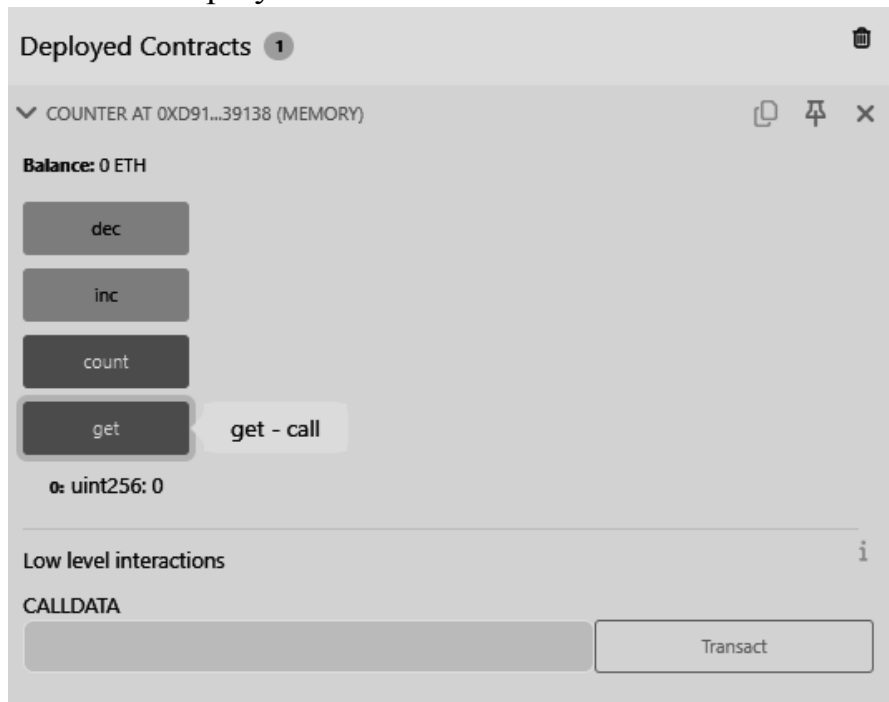
- Ether and Wei: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- Gas and Gas Price: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- Sending Transactions: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

Tutorial 1 - Compile the code



Tutorial 1-Deploy the contract



Tutorial 1-Increment

COUNTER AT 0XD91...39138 (MEMORY)

Balance: 0 ETH

dec

inc

count

get

get - call

0: uint256: 3

Low level interactions

CALLDATA

Transact

Tutorial 1-Decrement

COUNTER AT 0XD91...39138 (MEMORY)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 1

Low level interactions

CALLDATA

Transact

Tutorial 2

REMUX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH

Tutorials list

2. Basic Syntax 2 / 19

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

★ **Assignment**

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     //Niraj Kothawade
7     string public name = "Alice";
8 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

data: 0x6d4...ce63c

Activate Windows Go to Settings to activate Windows.

Tutorial 3

LEARNETH

Tutorials list

3. Primitive Data Types 3 / 19

Structs

[Watch a video tutorial on Primitive Data Types.](#)

★ **Assignment**

1. Create a new variable `newAddr` that is a `public` `address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Primitives {
5     bool public boo = true;
6
7     // Niraj Kothawade
8     uint8 public u8 = 1;
9     uint public u256 = 456;
10    uint public u = 123; // uint is an alias for uint256
11
12    /*
13     Negative numbers are allowed for int types.
14     Like uint, different ranges are available from int8 to int256
15     */
16    int8 public i8 = -1;
17    int public i256 = 456;
18    int public i = -123; // int is same as int256
19
20    address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
21 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Welcome to Remix 1.5.1

Activate Windows Go to Settings to activate Windows.

Tutorial 4

Tutorials list

Syllabus

4. Variables

4 / 19

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

- Create a new public state variable called `blockNumber`.
- Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // Niraj Kothawade
6     // State variables are stored on the blockchain.
7     string public text = "Hello";
8     uint public num = 123;
9     uint public blockNumber;
10
11     function doSomething() public { 22334 gas
12         // Local variables are not saved to the blockchain.
13         uint i = 456;
14
15         // Here are some global variables
16         uint timestamp = block.timestamp; // Current block timestamp
17         address sender = msg.sender; // address of the caller
18         blockNumber = block.number;
19     }
20 }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 5

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable

5 / 19

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

★ Assignment

- Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
- Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // Niraj Kothawade
6     // State variable
7     uint public num;
8     bool public b;
9
10    // You need to send a transaction to write to a state variable.
11    function set(uint _num) public { 22536 gas
12        num = _num;
13    }
14
15    // You can read from a state variable without sending a transaction.
16    function get() public view returns (uint) { 2475 gas
17        return num;
18    }
19
20    function get_b() public view returns (bool) { 2539 gas
21        return b;
22    }
23 }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 6

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Niraj Kothawade
8     // Promise not to modify the state.
9     function addToX(uint y) public view returns (uint) {
10         return x + y;
11     }
12
13     // Estimated execution cost: infinite gas
14
15     function addToX2(uint y) public {
16         x = x + y;
17     }
18 }
```

Explain contract

AI copilot

0

☐ Listen on all transactions

Filter with transaction hash or ad...

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 7

Tutorials list **Syllabus**

5.3 Functions - Modifiers and Constructors 7 / 19

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

[Watch a video tutorial on Function Modifiers.](#)

★ Assignment

- Create a new function, `increaseX`, in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
- Make sure that `x` can only be increased.
- The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract FunctionModifier {
5     // We will use these variables to demonstrate how to use
6     // modifiers.
7     address public owner;
8     uint public x = 10;
9     bool public locked;
10    // Niraj Kothawade
11
12    constructor() {
13        // Set the transaction sender as the owner of the contract.
14        owner = msg.sender;
15    }
16
17    // Modifier to check that the caller is the owner of
18    // the contract.
19    modifier onlyOwner() {
20        require(msg.sender == owner, "Not owner");
21        // Underscore is a special character only used inside

```

✎ Explain contract

AI copilot ON

0 ☐ Listen on all transactions
🔍 Filter with transaction hash or ad...
🔄 🖱️ ✕

Welcome to Remix 1.5.1
Activate Windows

Go to Settings to activate Windows.

Tutorial 8

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs
8 / 19

that are publicly visible." From the [Solidity documentation](#).

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute. Watch a video tutorial on [Function Outputs](#).

★ Assignment

Create a new function called `returnTwo` that returns the values `2` and `true` without using a return statement.

Check AnswerShow answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Function {
5     // Niraj Kothawade
6     // Functions can return multiple values.
7     function returnMany() infinite gas
8     {
9         public
10        pure
11        returns (
12            uint,
13            bool,
14            uint
15        )
16    {
17        return (1, true, 2);
18    }
19
20    // Return values can be named.
21    function named() infinite gas
22    {
23        public
24    }
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or ad...

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 9

Tutorials list

Syllabus

6. Visibility
9 / 19

contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on [Visibility](#).

★ Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check AnswerShow answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Base {
5     // Niraj Kothawade
6     // Private function can only be called
7     // - inside this contract
8     // Contracts that inherit this contract cannot call this function.
9     function privateFunc() private pure returns (string memory) { infinite gas
10         return "private function called";
11     }
12
13     function testPrivateFunc() public pure returns (string memory) { infinite gas
14         return privateFunc();
15     }
16
17     // Internal function can be called
18     // - inside this contract
19     // - inside contracts that inherit this contract
20     function internalFunc() internal pure returns (string memory) { infinite gas
21         return "internal function called";
22     }
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or ad...

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 10

Tutorials list

Syllabus

7.1 Control Flow - If/Else

10 / 19

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

[Watch a video tutorial on the If/Else statement.](#)

★ Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     //Niraj Kothawade
6     function foo(uint x) public pure returns (uint) { infinite gas
7         if (x < 10) {
8             return 0;
9         } else if (x < 20) {
10            return 1;
11        } else {
12            return 2;
13        }
14    }
15
16    function ternary(uint _x) public pure returns (uint) { infinite gas
17        // if (_x < 10) {
18        //     return 1;
19        // }
20        // return 2;
21    }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 11

Tutorials list

Syllabus

7.2 Control Flow - Loops

11 / 19

iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

[Watch a video tutorial on Loop statements.](#)

★ Assignment

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5     //Niraj Kothawade
6     uint public count;
7     function loop() public { infinite gas
8         // for loop
9         for (uint i = 0; i < 10; i++) {
10             if (i == 5) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18             count++;
19         }
20     }
21     // while loop
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 12

Tutorials list

Syllabus

8.1 Data Structures - Arrays
12 / 19

can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Array {
5     // Niraj Kothawade
6     // Several ways to initialize an array
7     uint[] public arr;
8     uint[] public arr2 = [1, 2, 3];
9     // Fixed sized array, all elements initialize to 0
10    uint[10] public myFixedSizeArr;
11    uint[3] public arr3 = [0, 1, 2];
12
13    function get(uint i) public view returns (uint) {
14        return arr[i];
15    }
16
17    // Solidity can return the entire array.
18    // But this function should be avoided for
19    // arrays that can grow indefinitely in length.
20    function getArr() public view returns (uint[3] memory) {
21        return arr3;
22    }
23 }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 13

Tutorials list

Syllabus

8.2 Data Structures - Mappings
13 / 19

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

★ Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // Niraj Kothawade
6     // Mapping from address to uint
7     mapping(address => uint) public balances;
8
9     function get(address _addr) public view returns (uint) {
10        // Mapping always returns a value.
11        // If the value was never set, it will return the default value.
12        return balances[_addr];
13    }
14
15    function set(address _addr) public {
16        // Update the value at this address
17        balances[_addr] = _addr.balance;
18    }
19
20    function remove(address _addr) public {
21        // Reset the value to the default value.
22    }
23 }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 14

Tutorials list

Syllabus

8.3 Data Structures - Structs14 / 19

members and update a struct. We initialize an empty struct instance then update its member by assigning it a new value (line 23).

Accessing structs

To access a struct's member we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check AnswerShow answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract Todos {
5     // Miraj Kothawade
6     struct Todo {
7         string _text;
8         bool completed;
9     }
10
11     // An array of 'Todo' structs
12     Todo[] public todos;
13
14     function create(string memory _text) public {
15         // 3 ways to initialize a struct
16         // - calling it like a function
17         todos.push(Todo(_text, false));
18
19         // key value mapping
20         todos.push(Todo({text: _text, completed: false}));
21     }
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

Welcome to Remix 1.5.1

Activate Windows

Go to Settings to activate Windows.

Tutorial 15

Tutorials list

Syllabus

8.4 Data Structures - Enums15 / 19

update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check AnswerShow answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Enum representing shipping status
6     // Miraj Kothawade
7     enum Status {
8         Pending,
9         Shipped,
10        Accepted,
11        Rejected,
12        Canceled
13    }
14
15     enum Size {
16         S,
17         M,
18         L
19     }
20
21     // Default value is the first element listed in
```

Explain contract

AI copilot

0 Listen on all transactions

Filter with transaction hash or address

Welcome to Remix 1.5.1

Activate Windows

Go to Settings to activate Windows.

Tutorial 16

Tutorials list

Syllabus

9. Data Locations

16 / 19

★ Assignment

- Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
- Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
- Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
- Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     //Niraj Kothawade
6     uint[] public arr;
7     mapping(uint => address) map;
8     struct MyStruct {
9         uint foo;
10    }
11    mapping(uint => MyStruct) public myStructs;
12
13    function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory){
14        // call _f with state variables
15        _f(arr, map, myStructs[1]);
16        // get a struct from a mapping
17        MyStruct storage myStruct = myStructs[1];
18        myStruct.foo = 4;
19        // create a struct in memory
20        MyStruct memory myMemStruct = MyStruct(0);
21        MyStruct memory myMemStruct2 = myMemStruct;
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 17

Tutorials list

Syllabus

10.1 Transactions - Ether and Wei

17 / 19

`gwei`
One `gwei` (giga-wei) is equal to 1,000,000,000 (10⁹) `wei`.

`ether`
One `ether` is equal to 1,000,000,000,000,000 (10¹⁸) `wei` (line 11).
[Watch a video tutorial on Ether and Wei.](#)

★ Assignment

- Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
- Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10⁹.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     // Niraj Kothawade
6     uint public oneWei = 1 wei;
7     // 1 wei is equal to 1
8     bool public isOneWei = 1 wei == 1;
9
10    uint public oneEther = 1 ether;
11    // 1 ether is equal to 10^18 wei
12    bool public isOneEther = 1 ether == 1e18;
13
14    uint public oneGwei = 1 gwei;
15    // 1 ether is equal to 10^9 wei
16    bool public isOneGwei = 1 gwei == 1e9;
17 }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 18

10.2 Transactions - Gas and Gas Price
18 / 19

of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     // Niraj Kothawade
6     uint public i = 0;
7     uint public cost = 170367;
8
9     // Using up all of the gas that you send causes your transaction to fail.
10    // State change
11    // Gas spent at
12    function forever() {
13        // Here we run a loop until all of the gas are spent
14        // and the transaction fails
15        while (true) {
16            i += 1;
17        }
18    }
19 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Tutorial 19

10.3 Transactions - Sending Ether
19 / 19

create a new contract on sending ether.

★ Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ReceiveEther {
5     // Niraj Kothawade
6     /*
7      * Which function is called, fallback() or receive()?
8      *
9      * send Ether
10     * msg.data is empty?
11     * yes \ no
12     * yes \ no
13     * receive() exists? fallback()
14     * yes \ no
15     * receive() fallback()
16     */
17 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or ad...

Welcome to Remix 1.5.1

Activate Windows
Go to Settings to activate Windows.

Conclusion:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.